NumPy 常用函数速查

一、基本数学（数学运算）

函数	作用	示例
`np.sqrt(x)`	开平方	`np.sqrt(9)` → 3.0
`np.exp(x)`	e 的 x 次方	`np.exp(1)` ≈ 2.718
`np.log(x)`	自然对数（底为 e）	`np.log(np.e)` → 1.0
`np.sin(x)`	正弦（弧度）	`np.sin(np.pi/2)` → 1.0 (pi指的是Π)
`np.abs(x)`	绝对值	`np.abs(-3)` → 3
`np.power(a, b)`	幂运算： a^b	`np.power(2, 3)` → 8
`np.round(x, n)`	四舍五入，保留 n 位小数	`np.round(3.1415, 2)` → 3.14
`np.ceil(x)`	向上取整	`np.ceil(2.34)` → 3
`np.floor(x)`	向下取整	`np.floor(2.99)` → 2

二、统计（聚合计算）

函数	作用
`np.sum(x)`	求和
`np.mean(x)`	平均值（用'%.3f'%保留三位小数）
`np.median(x)`	中位数
`np.std(x)`	标准差
`np.var(x)`	方差 （$\sigma^2 = \Sigma(x_i - \mu)^2 / N$）
`np.min(x)`	最小值
`np.max(x)`	最大值
`np.argmin(x)`	最小值索引
`np.argmax(x)`	最大值索引
`np.percentile(x, q)`	第 q 分位数
`np.cum(x)`	累积和
`np.cumsum(x)`	累积积

> 💡 默认对整个数组求聚合；想按行/按列，加 `axis=1` （行）或 `axis=0` （列）。

三、比较（返回布尔数组）

函数	作用	示例
`np.greater(a, b)`	a > b	返回逐元素比较结果的数组

函数	作用	示例
np.less(a, b)	a < b	同上
np.equal(a, b)	a == b	同上
np.logical_and(a, b)	a & b （与）	用于布尔数组
np.where(条件, x, y)	三元运算	条件为真取 x，否则取 y

> 💡 a > b 直接写也可以； np.where 相当于向量版的 if else ，非常常用。

四、其他（通用工具）

函数	作用	示例
np.concatenate((a, b))	拼接两个数组	默认按行拼接，加 axis=1 按列
np.split(x, indices)	切分数组	np.split(x, [2]) 在索引为 2 的前面切 np.split(x, 2)切出两份（必须要能等分）
np.reshape(x, shape)	改变形状	np.reshape(x, (2,3))
np.copy(x)	复制数组（深拷贝）	改副本不影响原数组
np.isnan(x)	判断是否为 NaN	返回布尔数组

五、去重

函数	作用	示例
np.unique(x)	去重并排序（从小到大）	np.unique([3,1,2,2]) → [1,2,3]
np.inld(a, b)	判断 a 的每个元素是否在 b 中	返回布尔数组

六、排序

函数	作用	备注
np.sort(x)	返回排序后的副本	不改原数组
x.sort()	原地排序	直接改原数组
np.argsort(x)	返回排序后的下标	用于"间接排序"
np.lexsort(keys)	按多个键排序	最后一个键是主键

速记口诀

- **基本数学**：单元素级别的运算
- **统计**：把一堆数压成一个数（聚合）
- **比较**：判断真假，返回布尔数组
- **其他**：拼接、切分、变形、复制
- **去重**：留下不重复的

- **排序**：把元素按大小排好

> ⚠️ 几乎所有函数都有重要参数 `axis`：0 是"沿行"（跨行计算，每列一个结果），1 是"沿列"（跨列计算，每行一个结果）。

In [ ]:
```
#四舍五入
import numpy as np
print(np.round([1.2,1.4,1.5,12.5]))
#如果有.5000000则看整数部分，奇数进一位，偶数退一位（如上）
```

In [ ]:
```
#检测缺失值
np.isnan([1.2,1.4,np.nan,12.5])
#空的地方是true，有数据的地方是false
#用于导出excel时检查是否有没有填写的地方
#不用写print
```

In [ ]:
```
#累计和/积
arr = np.array([1,2,3,4])
print(np.sum(arr))
print(np.cumsum(arr))#第n项是arr中前n项的和
print(np.cumprod(arr))#第n项是arr中前n项的积
```

In [ ]:
```
#比较（返回布尔，要加print）
print(np.greater(a,b))
#a/b可以是数也可以是数列，数列和数列是对应位置一一比较
```

In [ ]:
```
#逻辑比较
a = np.array([1,1,0])
b = np.array([2,0,0])
#都是指相同位置的地方
print(np.logical_and(a, b))#都是true为true，其他为false
print(np.logical_not(a))#0为true，其他是false
print(np.logical_or(a, b))#有一个true就是true
```

In [ ]:
```
#检查元素是否至少有一个元素为true
print(np.any([0,0,1]))
#检查元素是否全为true
print(np.all([1,1,0,1]))
```

In [ ]:

```
#自定义条件
arr = np.array([2,3,4,5,8])
print(np.where(arr))#原因：当 np.where() 只传一个参数时，它把数组当成条件，返回所有为真（True）元素的下标。而 [2,3,4,5,8] 里每个数都是非零 = 真，所以 5 个数全入选，下标就是 [0,1,2,3,4]
print(np.where(arr>=4,1,0))
#括号里依次是条件，满足条件生成的，不满足条件生成的
print(np.where(arr>4,1,np.where(arr==4,2,3)))#嵌套
#优化版
print(np.select([arr<4,(arr>4) & (arr<1000),arr==4],
                [3,2,1],
                default=0))
#注意先条件后结果，用&的话左右要加小括号，default的输出类型要和另外几个一样。格式复杂，别写错了。换行的话更好看（不强制）
```

In [ ]:

```
# 排序函数
np.random.seed(0)
arr = np.random.randint(1, 100, 20)
print(arr)
# arr.sort()  会返回none，会改变原有数组
# print(arr)
print(np.sort(arr))#不改变数组
print(np.argsort(arr))#输出的是排序后对应位置上的索引
print(arr)
```